Claude Code 源码深度拆解报告

4756 文件架构全景 × Vibe Claw 可借鉴清单

> 基于 `@anthropic-ai/claude-code@2.1.88` source map 还原分析

一、文件分布全景

目录	文件数	核心职责
`utils/`	571	工具函数：git、权限、模型、shell、环境
`components/`	389	Ink (终端 React) UI 组件
`commands/`	207	40+ 个 slash 命令实现
`tools/`	184	30+ 个 AI 可调用工具
`services/`	130	API、MCP、分析、Session Memory
`hooks/`	104	生命周期钩子（权限、UI、状态）
`skills/`	20	Skill 加载/注册/bundled skills
`tasks/`	12	Agent 任务生命周期管理
`coordinator/`	1	多 Agent 编排模式（18922 chars 单文件）
`memdir/`	8	持久化记忆系统 (MEMORY.md)
`context/`	11	React Context 状态管理

二、你的 Skill 缺什么 — 协议层 vs 执行层的鸿沟

你的直觉完全正确：**Skill 是协议，但协议不会自己跑。** 从源码看，Claude Code 在”让协议真的动起来”这件事上，做了五层工程：

第 1 层：Agent 不是 Prompt，是 Runtime Object

你的 Skill 里 SAM/SCOUT/FORGE 是角色描述。Claude Code 的 Agent 是一个有类型定义的运行时对象：

```
// 源码：AgentDefinition (loadAgentsDir.ts)
type AgentDefinition = {
  agentType: string              // 角色 ID
  description: string            // 角色描述
  tools: string[]                // 工具白名单 ['Read', 'Edit', 'Bash'] 或 ['*']
  disallowedTools?: string[]     // 工具黑名单
  model?: string                 // 可指定模型
  mcpServers?: AgentMcpServerSpec[]  // 可接入 MCP
  hooks?: HooksSettings          // 生命周期钩子
  memoryScope?: 'user' | 'project' | 'local'  // 记忆范围
  maxTurns?: number              // 防无限循环
  permissionMode?: PermissionMode // 权限模式
  getSystemPrompt: () => string  // 系统提示词
  source: 'built-in' | 'user' | 'plugin'
  baseDir: string                // 基础目录（skill 文件所在位置）
}
```

可借鉴：你的 Skill 里的 `skills.yaml` 白名单思路是对的，但需要升级成每个 agent 独立声明自己的工具权限，而不是项目级统一白名单。

第 2 层：Task 生命周期是硬编码的状态机

```
// 源码：Task.ts
type TaskStatus = 'pending' | 'running' | 'completed' | 'failed' | 'killed'

type TaskStateBase = {
  id: string           // 带类型前缀：a- (agent), b- (bash), t- (teammate)
  type: TaskType       // 'local_agent' | 'in_process_teammate' | ...
  status: TaskStatus
  startTime: number
  endTime?: number
  outputFile: string   // 每个 task 有独立的输出文件
  outputOffset: number // 增量读取偏移
  notified: boolean    // 是否已通知 coordinator
}
```

你的 Skill 缺这个：Session 生命周期（开始→执行→收尾）是文档化的，但没有被编码成状态机。`vibe-start` 到 `vibe-close` 之间没有东西追踪"当前处于哪个阶段、哪些步骤已完成"。

第 3 层：Forked Agent — 子 Agent 的真实执行机制

这是最关键的，直接回答你"子 agent 怎么真的动起来"的问题：

```
// 源码：forkedAgent.ts — 子 agent 的执行入口
type ForkedAgentParams = {
  promptMessages: Message[]          // 初始 prompt（主 agent 注入上下文）
  cacheSafeParams: CacheSafeParams   // 共享父级 prompt cache
  canUseTool: CanUseToolFn           // 权限检查函数
  maxTurns?: number                  // 最大轮次
  onMessage?: (msg: Message) => void // 流式回调
}

// CacheSafeParams — 父子 agent 共享 prompt cache 的关键
type CacheSafeParams = {
  systemPrompt: SystemPrompt     // 系统提示词
  userContext: Record<string, string>  // 用户上下文
  toolUseContext: ToolUseContext  // 工具+模型+选项
  forkContextMessages: Message[] // 父级上下文消息（用于 cache 命中）
}
```

核心发现：子 agent 并不是"重新开一个 Claude 对话"，而是一个 **forked query loop**：

1. 继承父级的 system prompt 和 prompt cache（省 token）
2. 拿到自己的 `promptMessages` （主 agent 注入的任务描述）
3. 有独立的 `AbortController` （可被叫停）
4. 有独立的 `FileStateCache` （文件状态隔离）
5. 每个 tool call 的结果通过 `onMessage` 回调流式上报

对你的启示：如果用路线二（OpenClaw 编排），每次 `claude --agent scout` 调用本质上就是创建一个 ForkedAgent。它的上下文不是靠"重读记忆文件"，而是靠**主 agent 在 prompt 里注入精华上下文**。记忆文件是持久化用的，不是运行时注入用的。

第 4 层：三层记忆系统

Claude Code 有三个独立的记忆机制，各管各的：

```
┌──────────────────────────────────────────────────┐
│                 记忆系统全景                      │
│                                                   |
│  1. MEMORY.md (memdir/)                           │
│     ├── 位置：~/.claude/MEMORY.md（用户级）       |
│     │          .claude/MEMORY.md（项目级）        |
```

```
│      ├── 容量：最大 200 行 / 25KB                  │
│      ├── 注入：每次对话开始时自动注入 system prompt │
│      └── 对标：你的 PROJECT.md                     │
│                                                    │
│  2. Session Memory (services/SessionMemory/)       │
│      ├── 触发：context window 超过 10000 token     │
│      ├── 更新：每增长 5000 token 或 3 次工具调用    │
│      ├── 实现：后台 forked agent 提取摘要           │
│      ├── 输出：session-memory.md 文件               │
│      └── 对标：你的 CURRENT_STATE.md + HANDOFF      │
│                                                    │
│  3. Agent Memory (tools/AgentTool/agentMemory)     │
│      ├── 范围：user / project / local 三级          │
│      │   user:    ~/.claude/agent-memory/<type>/   │
│      │   project: .claude/agent-memory/<type>/     │
│      │   local:   .claude/agent-memory-local/<type>/│
│      ├── 每个 agent 类型有独立记忆目录               │
│      └── 对标：你没有这个──每个 agent 的独立记忆     │
│                                                    │
│  + Scratchpad (coordinator 模式)                   │
│      ├── 所有 worker 共享的临时目录                 │
│      ├── 读写不需要权限确认                         │
│      └── 对标：你的四文档体系的"磁盘交换"版本       │
└────────────────────────────────────────────────────┘
```

关键差距：你的 Skill 有四文档体系但没有：

- 自动摘要（Session Memory 是后台 agent 自动提取的）
- Agent 级独立记忆（每个角色有自己的记忆空间）
- 容量管理（MEMORY.md 有硬限制 200 行 / 25KB）

第 5 层：Hooks — 生命周期的硬执行点

```
// 源码：hooks/settings/types.ts
type HookCommand =
  | { type: 'command'; command: string; shell?: string; if?: string }
  | { type: 'prompt'; prompt: string; if?: string }
  | { type: 'agent'; prompt: string; if?: string }
  | { type: 'http'; url: string; if?: string }

// Hook 触发点
type HookEvent =
  | 'PreToolCall'      // 工具调用前
  | 'PostToolCall'     // 工具调用后
```

```
  | 'Notification'      // 通知
  | 'Stop'             // 停止
  | 'SubagentStop'     // 子 agent 停止
```

这解决你的核心痛点：你说"没办法监控 agent 是不是真的懂这个任务"——Hooks 就是答案。你可以在 `PostToolCall` 里挂一个检查脚本，验证 agent 的输出是否符合预期。比如 FORGE 写完代码后，自动触发 lint + type check。

三、Claude Code 的 30+ 个工具清单

从 `constants/tools.ts` 和各工具目录提取：

核心工具（Agent 可用）

工具名	职责	Agent 可用?
`Read`	读文件	✅
`Edit`	编辑文件（精确替换）	✅
`Write`	写新文件	✅
`Bash`	执行 shell 命令	✅
`Glob`	文件名模式搜索	✅
`Grep`	文件内容搜索	✅
`NotebookEdit`	Jupyter notebook 编辑	✅
`WebSearch`	网页搜索	✅
`WebFetch`	获取网页内容	✅
`TodoWrite`	写 TODO 列表	✅
`Skill`	调用已注册的 skill	✅
`ToolSearch`	搜索可用工具	✅

协调工具（仅 Coordinator / 主 agent）

工具名	职责	Agent 可用?
`Agent`	创建子 agent	❌（子 agent 不可嵌套）
`SendMessage`	向子 agent 发消息	❌
`TaskStop`	停止子 agent	❌
`AskUserQuestion`	向用户提问	❌
`TaskOutput`	读取 task 输出	❌
`EnterPlanMode`	进入规划模式	❌
`ExitPlanMode`	退出规划模式	✅（仅 plan agent）
`SyntheticOutput`	合成输出（内部用）	特殊

Swarm / 团队工具

工具名	职责
`TeamCreate`	创建团队
`TeamDelete`	删除团队
`TaskCreate`	创建共享任务
`TaskGet`	获取任务状态
`TaskList`	列出所有任务
`TaskUpdate`	更新任务
`EnterWorktree`	进入 Git worktree
`ExitWorktree`	退出 Git worktree

四、14 个 Bundled Skills 详解

Skill	功能	你能学什么
	从当前 session 提炼	**直接用**：每次好的 vibe session

skillify	可复用 skill	结束跑一次
verify	验证代码变更	对标你的 Evaluator 四维评估
verifyContent	验证内容（非代码）	可用于验证文档更新质量
batch	批量执行操作	对标你的 AUX eval 跑批
loop	循环执行直到条件满足	**新概念**：你的 Skill 没有重试/循环机制
debug	调试模式	可用于 session 中的问题诊断
stuck	被卡住时的自救	**新概念**：你的 anti-patterns 可以增加”卡住检测”
remember	主动存储记忆	对标你的 INK 文档更新
simplify	简化代码/方案	可集成到 LENS 漂移检测后的修复建议
keybindings	快捷键配置	工程化辅助
claudeInChrome	浏览器集成	特定场景
claudeApi / claudeApiContent	API 内容生成	特定场景
scheduleRemoteAgents	远程 agent 调度	未来 OpenClaw 远程部署可参考
updateConfig	更新配置	工程化辅助

五、Skill 可直接借鉴的 7 个工程模式

1. Verification 的”两个 Failure Pattern”

```
你当前的 Evaluator 四维评估：功能完整性/接口稳定性/无后处理堆叠/中间态可观测

Claude Code 的 Verification Agent 增加了两个自我认知：
① Verification Avoidance：读代码→叙述测试→写 PASS，但从没跑过
② 被前 80% 诱惑：看到漂亮 UI 就想 PASS

→ 建议在 anti-patterns.md 中增加这两个模式
→ 建议 Evaluator 增加"是否实际执行了验证命令"的检查项
```

2. Session Memory 的自动摘要机制

```
你当前的做法：手动在 session 结束时更新 CURRENT_STATE.md

Claude Code 的做法：
- context window 超过 10000 token 后自动启动后台 agent 提取摘要
- 每增长 5000 token 或每 3 次工具调用更新一次
- 摘要存为独立文件，不污染主对话

→ 建议在 Skill 中增加"自动状态快照"触发条件
→ 不依赖 session 结束才更新，而是 session 中间就持续更新
```

3. Agent 级独立记忆目录

```
你当前的做法：所有 agent 共享 PROJECT.md / CHANGELOG.md

Claude Code 的做法：
- 每个 agent type 有独立记忆目录：.claude/agent-memory/<type>/
- 三个范围：user（跨项目）、project（本项目）、local（不入 VCS）

→ 建议增加：
  .vibe/agent-memory/
  ├── scout/    # SCOUT 历次扫描结果的经验积累
  ├── forge/    # FORGE 编码决策的模式记忆
  ├── lens/     # LENS 发现的漂移模式库
  └── ink/      # INK 文档更新的格式学习
```

4. `/loop` — 重试直到成功

```
你当前的做法：线性执行，失败就报告

Claude Code 的 /loop skill：
- 执行一个操作
- 检查是否满足条件
- 不满足就重试，最多 N 轮

→ 建议在 Sprint Contract 中增加 retry_policy：
  sprint:
    task: "实现用户登录"
    success_criteria: "所有测试通过"
    retry_policy:
      max_attempts: 3
      on_failure: "分析失败原因后重试"
```

5. `/stuck` — 卡住自救协议

```
你当前没有的：agent 被卡住时的处理流程

Claude Code 的 /stuck skill：
- 检测当前状态
- 分析为什么卡住
- 建议替代方案

→ 建议在 session-protocol.md 中增加"卡住检测"：
  如果同一个文件被修改 > 3 次且 eval 仍未通过：
  → 触发卡住协议
  → 回退到上一个稳定状态
  → 重新规划实现路径
```

6. Hooks 作为硬检查点

```
你当前的做法：全靠 prompt 约束（"修改 harness/ 需要用户确认"）

Claude Code 的做法：
  hooks:
    PreToolCall:
      - type: command
        command: "./scripts/check-harness.sh"
        if: "Edit(harness/*)"      # 只在编辑 harness 目录时触发
    PostToolCall:
      - type: command
        command: "npm run lint"
        if: "Edit(src/*)"          # 每次编辑源码后自动 lint

→ 建议在 Skill 中增加 hooks 定义：
  hooks:
    post_forge_edit:
      command: "npx tsc --noEmit"  # FORGE 改完代码自动 type check
    pre_harness_change:
      command: "echo '⚠️ HARNESS CHANGE DETECTED' && exit 1"
```

7. Scratchpad — 跨 Agent 共享工作区

```
你当前的做法：通过四文档体系间接共享

Claude Code 的做法：
```

```
- Coordinator 模式下有 scratchpad 目录
- 所有 worker 可读写，不需要权限确认
- 用于"不值得放进正式记忆但 worker 间需要共享的临时信息"

→ 建议增加 .vibe/scratchpad/
  ├── scout-findings.md    # SCOUT 本次扫描的临时发现
  ├── forge-progress.md    # FORGE 当前进度
  └── lens-concerns.md     # LENS 发现但还不确定的问题
```

六、你的核心问题的答案

“Skill 给到 OpenClaw 去运作，它还是没办法学会”

对的。 因为你的 Skill 是”协议文档”，不是”可执行规范”。区别在于：

维度	你的 Skill 现在	Claude Code 的实现
Agent 定义	自然语言角色描述	TypeScript 类型 + runtime object
工具权限	项目级白名单	每个 agent 独立声明
状态追踪	文档更新	TaskState 状态机
质量检查	prompt 约束	Hooks 硬检查点
记忆管理	手动更新文件	自动摘要 + 三层记忆
失败处理	无	/loop 重试 + /stuck 自救

“包括那几个子 agent 也都要调用 CLI 或者具体的 API”

完全正确。 从源码看，每个子 agent 的执行链是：

```
1. Coordinator 调用 Agent tool
2. Agent tool 创建 ForkedAgentParams（包含 prompt、工具列表、模型）
3. forkedAgent.ts 创建子 query loop
4. 子 agent 在 loop 中调用实际工具（Read/Edit/Bash/...）
5. 每个工具调用经过 hooks 检查
6. 完成后通过 <task-notification> 回报 Coordinator
```

你要在 OpenClaw 中复刻这个，需要的不是更好的 Skill 文档，而是：

```
SAM (n8n workflow / 持续运行的 Claude 实例)
```

```
│
├── 调用：claude code --agent scout --message "<注入的上下文>"
│      └── scout 内部：读文件 → 分析 → 返回 JSON 格式报告
│
├── SAM 解析报告，判断质量，决定下一步
│
├── 调用：claude code --agent forge --message "<SAM 合成的 spec>"
│      └── forge 内部：编码 → 自测 → 提交 → 返回结果
│
└── 调用：claude code --agent lens --message "<forge 改了什么>"
       └── lens 内部：读代码 → 跑测试 → 验证 → 返回报告
```

每个 `claude code --agent` 调用就是一个 ForkedAgent。 它自带工具、自带权限、自带 context。SAM 不需要教它"怎么用工具"——工具能力是 Claude Code CLI 内置的。SAM 只需要告诉它"做什么"以及"做到什么程度算完成"。

七、Skill v3 升级建议

基于以上分析，Vibe Claw Skill 的下一个版本应该增加：

1. Agent 定义规范化

```yaml
# .vibe/agents/scout.yaml
name: scout
type: read-only
tools: [Read, Glob, Grep, Bash]
memory_scope: project
max_turns: 20
hooks:
  post_completion:
    - type: command
      command: "python scripts/validate_scan_report.py"
inputs:
  - SESSION_HANDOFF.md
  - PROJECT.md
  - CURRENT_STATE.md
outputs:
  - .vibe/scratchpad/scout-report.json
success_criteria:
  - "输出包含三层扫描结果"
  - "每层有明确的 ✅/⚠️/❌ 判定"
```

2. 状态机编码

```yaml
# .vibe/session-state.yaml
current_phase: idle
phases:
  idle → scanning:
    trigger: vibe-start
    agent: scout
  scanning → contracting:
    trigger: scout-complete
    requires: scout-report.json exists
  contracting → executing:
    trigger: user-confirms-sprint
    agent: forge
  executing → reviewing:
    trigger: forge-complete
    agent: lens
  reviewing → closing:
    trigger: lens-approved
    agent: ink
  closing → idle:
    trigger: ink-complete
```

3. 自动摘要触发

```yaml
# .vibe/auto-summary.yaml
triggers:
  token_threshold: 10000    # context 超过 10000 token 启动
  update_interval: 5000     # 每增长 5000 token 更新
  tool_call_interval: 3     # 每 3 次工具调用更新
output: .vibe/scratchpad/session-summary.md
```

4. 失败处理协议

```yaml
# .vibe/failure-policy.yaml
stuck_detection:
  same_file_edits: 3        # 同一文件改了 3 次
  eval_failures: 2           # eval 连续失败 2 次
  action: rollback_and_replan

retry_policy:
  max_attempts: 3
  backoff: exponential
  on_max_failures: escalate_to_user
```

这些 YAML 文件既可以被 Claude 当 prompt 读（路线一），也可以被 n8n 当配置解析（路线二）。**同一份规范，两种消费者。**